

QRCS Corporation

ChaCha Stream eXtended (CSX) Analysis

Title: Implementation Analysis of the ChaCha Stream Extended (CSX) stream cipher

Author: John G. Underhill

Institution: Quantum Resistant Cryptographic Solutions Corporation (QRCS)

Date: November 2025

Document Type: Technical Cryptanalysis Report

Revision: 1.0

Chapter 1: Introduction

1.1 Abstract

This paper presents a formal cryptanalysis of the CSX Authenticated Encryption with Associated Data (AEAD) stream cipher, developed by Quantum Resistant Cryptographic Solutions (QRCS). CSX is a high-performance, symmetric stream cipher designed to provide both encryption and authentication using a separate ARX stream cipher and an independent Keccak-based MAC. The cipher integrates a Keccak-derived sponge core for key expansion, with the nonce written directly into the ARX state, and employs bitwise mixing operations optimized for AVX/AVX2 vectorization. Its design emphasizes post-quantum resistance, constant-time execution, and side-channel resilience through deterministic memory access and SIMD-based masking.

The analysis in this paper is based on the complete `csx.c` and `csx.h` implementation files and their supporting modules (`memutils`, `sha3`), as provided by QRCS. All cryptanalytic assessments are verified against the provided reference implementation. This study evaluates the cipher's internal structure, permutation and mixing layers, initialization sequence, authentication tag construction, and its resistance to common cryptanalytic and implementation-level attacks. The goal is to determine whether CSX achieves its design objective of robust, high-speed, post-quantum AEAD security under realistic adversarial models.

1.1 Scope

This document is limited to the cryptographic analysis of the CSX AEAD cipher and its operational primitives. It includes examination of:

- Key and nonce scheduling functions.
- Internal state transformation, bit-mixing, and sponge absorption phases.
- Authentication tag derivation and verification mechanisms.
- Resistance to linear, differential, and algebraic attacks.
- Side-channel and fault analysis considerations.
- Implications of the Keccak-based initialization and its domain separation.

The study does **not** address performance metrics such as runtime or throughput, as those depend on implementation-specific optimizations. All analysis is conducted under the assumption of full adversarial access to ciphertexts, associated data, and oracle queries within the limits of standard IND-CPA and IND-CCA models.

1.2 Methodology

The analytical method follows a layered approach consistent with cryptanalytic standards:

1. **Algorithmic Review:** Extraction of all cryptographic primitives, constants, and operational parameters.
2. **Mathematical Modeling:** Formal description of the permutation and mixing layers using algebraic and Boolean function representations.
3. **Security Evaluation:** Testing against standard attack models (differential, linear, meet-in-the-middle, invariant subspace, cube, and algebraic).
4. **Implementation Validation:** Analysis of side-channel safety, constant-time memory operations, and SIMD-based resistance to timing leakage.
5. **Comparative Assessment:** Positioning relative to known AEAD constructions such as ChaCha20-Poly1305, AES-GCM, and Ascon.

1.3 Summary of Findings

Preliminary inspection of the code indicates that CSX employs:

- A 1024-bit state matrix partitioned into subblocks updated by mixed XOR-rotate-add operations.
- Key expansion via Keccak sponge functions producing independent cipher and MAC keys.
- Authentication is provided by a separate Keccak-based MAC instance operating independently of the keystream generator.
- Use of constant-time equality checks and secure memory clearing routines (in the C and C++ implementations), supporting resistance against timing and memory disclosure attacks.
- No variable-time branching or key-dependent memory access observed in the critical encryption/decryption loops.

The subsequent chapters will formalize these observations, beginning with a complete structural and mathematical description of the cipher core.

Chapter 2: Problem Definition and Analytical Scope

2.1 Objective

The objective of this analysis is to formally assess the cryptographic soundness of the CSX Authenticated Encryption with Associated Data (AEAD) stream cipher. The cipher's purpose is to provide simultaneous confidentiality and integrity in post-quantum secure communication environments, particularly where lightweight, high-throughput encryption is required.

This paper aims to determine whether the CSX design achieves resistance against classical and quantum adversarial models; specifically, indistinguishability under chosen-plaintext and chosen-ciphertext attacks (IND-CPA and IND-CCA), robustness of its authentication mechanism (INT-CTXT), and structural resistance against cryptanalytic exploitation of the internal permutation, key scheduling, and tag generation processes.

2.2 Theoretical Context

Authenticated stream ciphers are generally built around a central primitive that mixes key and nonce material into an evolving internal state, producing keystream words used

for encryption and tag generation. CSX follows this model but differs in that it couples a high-entropy, Keccak-based key expansion stage with a custom vectorized mixing core that operates on wide state registers. This architecture allows simultaneous encryption and authentication with minimal overhead.

The cipher's construction is deterministic for any key–nonce pair, eliminating nonce reuse ambiguity and enforcing strict uniqueness guarantees. From the specification and implementation files, it is evident that CSX combines a **sponge-based KDF** (derived from Keccak) with a **nonlinear feedback stream generator** that integrates both permutation and additive mixing components.

2.3 Problem Definition

From a cryptanalytic standpoint, the CSX cipher presents several distinct analysis domains:

1. Initialization and Key Mixing

- The cipher expands the user key through Keccak's absorption and squeezing phases; the nonce is written directly into the ARX state.
- A potential vulnerability could arise if the mixing function or domain separation constants fail to provide full diffusion before the first keystream block is emitted.
- The goal is to determine whether any weak-key or differential bias exists in the early rounds.

2. State Evolution and Permutation Layer

- The cipher's core consists of iterative rounds of XOR, rotate, and add operations operating over multiple words of the state.
- The analysis must determine whether these operations yield full bit-level diffusion within a minimal number of rounds.
- Resistance to differential-linear and algebraic correlation attacks is evaluated by modeling the round function's Boolean and modular structure.

3. Authentication Mechanism

- CSX computes its tag using an independent Keccak-based MAC that does not interact with the ARX encryption state.
- Authentication depends on the one-way accumulation of ciphertext blocks into a tag substate, combined with final Keccak squeezing.
- The analysis must determine whether this combined mechanism achieves existential unforgeability under chosen-ciphertext attack (EUF-CCA).

4. Side-Channel and Implementation Resistance

- The cipher makes extensive use of constant-time primitives from memutils.c and employs SIMD instructions under AVX and NEON environments.
- Analysis will verify constant-time properties and absence of key-dependent branching or memory access.
- The security implications of AVX512 and AVX2 optimization paths are considered, as these influence potential microarchitectural leakages.

5. Post-Quantum Implications

- Although CSX is a symmetric cipher and therefore not directly broken by quantum algorithms like Shor's or Grover's, the effective security margin must be evaluated under the assumption that Grover's algorithm provides a quadratic speed-up in key search.
- For an n -bit key, this reduces brute-force resistance from 2^n to approximately $2^{(n/2)}$. Therefore, key sizes and tag lengths are evaluated in that context.

2.4 Analytical Model

This analysis adopts a **black-box model** for the adversary with the following assumptions:

- The attacker can submit arbitrary plaintexts and nonces to the encryption oracle and observe corresponding ciphertexts and tags (IND-CPA model).
- The attacker may have limited decryption oracle access under a chosen-ciphertext setting, excluding the target message (IND-CCA model).

- The adversary knows the full algorithm and implementation (Kerckhoffs's principle) but not the secret key.
- The attacker's computational capabilities are bounded by current classical resources or polynomially enhanced quantum resources.

Mathematically, the CSX construction is modeled using two independent states.

Cipher initialization:

$$S_0 = S_0(K_c, N)$$

where K_c is the cSHAKE-derived cipher key and N is the nonce loaded into the ARX state.

Keystream generation:

for each block index i , the state S_i evolves only by incrementing the embedded counter and the keystream block is derived from the output of the 40-round permutation and feedforward. No associated data, plaintext, or ciphertext is absorbed into S_i .

MAC initialization and processing:

$$M_0 = M_0(K_{mc})$$

where K_{mc} is the cSHAKE-derived MAC key. The MAC state M absorbs the associated data A , its encoded length $\text{len}A$, the nonce N , the ciphertext C , and the total ciphertext length $\text{len}C$, then produces the tag T from the final MAC state.

The initialization, stream generation, MAC absorption, and MAC finalization are deterministic transformations that match the CSX implementation.

2.5 Analytical Scope

The study encompasses:

- **Primary focus:** Cryptographic structure, diffusion, and statistical resistance.
- **Secondary focus:** Implementation security and fault tolerance.
- **Excluded:** Performance profiling, software portability, and hardware-specific throughput measurements.

The CSX cipher's analysis is therefore bounded to its cryptographic design and mathematical robustness. The goal is not to optimize or benchmark performance but to validate or refute its claimed cryptographic security properties.

2.6 Limitations of the Analysis

The analysis assumes the reference implementation provided (csx.c and csx.h) corresponds to the official specification and that no alternate parameter sets or reduced-round configurations are deployed in practice. Because the cipher employs Keccak internally, its foundational sponge component inherits the 24-round Keccak permutation's well-studied security assumptions. The analysis therefore concentrates on the surrounding CSX layer; state mixing, key scheduling, and authentication tag integration, where unique design choices could affect resilience.

No empirical side-channel testing or physical fault injection is conducted; instead, the analysis verifies constant-time code properties and side-channel resistance from a theoretical standpoint.

Chapter 3: Model and Assumptions

3.1 Cipher Construction Summary

CSX is a keyed, synchronous stream cipher that operates on a 1024 bit internal state organized as a four-by-four matrix of 64 bit words. The cipher uses a 512 bit secret key and a 128 bit nonce. The state is initialized by copying the expanded cipher key, a fixed constant string, and the nonce into the matrix. After initialization, the state evolves only through a block counter and the fixed 40 round CSX permutation.

Authenticated encryption is achieved with a separate Keccak based message authentication code. During initialization, the input key is processed by cSHAKE 512 with a customization string. The first squeeze produces the cipher key used to construct the ARX state and the second squeeze produces the MAC key used to initialize either KMAC 512.. Encryption and authentication are composed using the encrypt then MAC method. The cipher generates the keystream independently of the MAC. The MAC authenticates associated data, nonce, ciphertext, and length fields.

The construction therefore consists of two components with strictly separated domains:

1. A ChaCha style ARX stream cipher with a 1024-bit state
2. A Keccak based MAC instance keyed with a distinct cSHAKE derived key

The ARX state does not absorb plaintext or ciphertext and does not act as the MAC key.

3.2 State Composition

The internal state is represented as sixteen 64-bit words. These words are filled during initialization as follows. The first eight words contain the 512-bit cipher key produced by cSHAKE 512. The next four words contain a fixed constant string embedded in the implementation. The final four words contain the 128-bit nonce in the first two positions and the remaining portion of the constant string in the last two positions.

Formally, let K_c be the 512-bit cipher key and let N be the 128-bit nonce. Let C be the fixed constant string. Then the state S is

$$S = (K_c[0], \dots, K_c[7], C[0], \dots, C[3], N[0], N[1], C[4], C[5])$$

No Keccak absorption of the key or nonce occurs in the initialization path. The state is populated directly from the cSHAKE derived cipher key and the fixed code string.

The block counter uses the last two state words as a 128-bit integer which is incremented by one for each 1024-bit block processed.

3.3 AEAD Model

CSX uses encrypt then MAC with a separate Keccak based MAC instance. The MAC key is derived from cSHAKE 512 through a second squeeze operation that is independent of the cipher key and not influenced by the ARX state. The MAC authenticates the associated data, its 32-bit length, the nonce, the ciphertext, and the 64-bit ciphertext length value accumulated across calls.

Let A be the associated data and let lenA be its length encoded as a 32-bit little endian integer. Let N be the nonce. Let C be the ciphertext and let lenC be the total number of ciphertext bytes processed, encoded as a 64-bit little endian integer. The MAC input is

$$\text{MAC input} = A \parallel \text{lenA} \parallel N \parallel C \parallel \text{lenC}$$

The ARX state does not contribute its value to the MAC computation. The final authentication tag is produced entirely from the MAC state.

This model matches the implementation and replaces earlier descriptions that treated the ARX state as part of the MAC input or assumed feedback of plaintext or ciphertext into the ARX permutation.

3.4 Key schedule and domain separation

Key expansion uses cSHAKE-512. Two independent squeezes derive distinct subkeys: one for the stream-cipher core and one for the MAC. The specification notes two separate Keccak squeeze calls to distance the XOF outputs. This report treats the two derived keys as independent for analysis.

KMAC-512 (24-round Keccak) provides the authentication primitive. Our security arguments for INT-CTXT will condition on the MAC choice and round count as specified.

3.5 Adversary capabilities

The adversary knows the full algorithm and implementation details (Kerckhoffs). Oracle access follows standard AEAD notions: chosen-plaintext queries to encryption with chosen nonce and AAD; chosen-ciphertext queries to decryption excluding the challenge target. No assumption is made about message structure beyond byte strings.

3.6 Usage conditions adopted by this report

This analysis conditions security on unique nonces per key for every encryption. The specification loads a 128-bit nonce into the state and uses a per-block counter to ensure block uniqueness; the report therefore treats nonce uniqueness as a usage requirement rather than a proven tolerance to reuse.

Associated data, if present, is input to the MAC prior to encryption or decryption as indicated by the API example.

3.7 Implementation assumptions

The analysis is based on the provided reference implementation set (From the QSC library: `csx.h/c`, `sha3.h/c`, `memutils.c/h`). Where side-channel properties are referenced, they derive from explicit routines in these sources. For example, the codebase provides secure memory clearing routines for state disposal. This report assumes such disposal is invoked as prescribed by the API.

No claim is made here about constant-time equality across all library primitives beyond what can be verified in code; side-channel discussion in later chapters is confined to the observed behaviors of these files.

3.8 Out-of-scope elements

Performance metrics and platform-specific acceleration claims (e.g., AVX2/AVX-512 paths) are not used as evidence of security. Comparative statements about other AEAD

schemes are restricted to parameter contrasts quoted from the specification where present.

Chapter 4: Related Work and Design Lineage

4.1 Historical context

Authenticated encryption with associated data (AEAD) has evolved from early constructions such as Encrypt-then-MAC schemes based on block ciphers, to integrated designs where encryption and authentication share an internal state. Among the best-known representatives of the latter group are ChaCha20-Poly1305, AES-GCM, and the Keccak-derived Ascon and Ketje families. CSX follows the AEAD model by combining a wide-state ARX cipher with a separate Keccak-based MAC. It combines a wide-state stream core similar in concept to ChaCha with a Keccak-based initialization and tag generation mechanism. This design places CSX conceptually between the permutation-based AEADs standardized by NIST and high-throughput ARX (add-rotate-xor) ciphers optimized for software vectorization.

The specification situates CSX as part of a family of symmetric primitives maintained within the QRCS library. The cipher was designed to act as a universal AEAD layer in post-quantum communication protocols where the key material itself originates from lattice or code-based exchanges. Its dependency on Keccak for key expansion and tag computation is deliberate: it leverages the well-studied diffusion and non-linearity properties of the Keccak permutation while preserving performance comparable to ARX stream ciphers on commodity processors.

4.2 Architectural influences

The initialization and key-derivation phase of CSX is derived from the SHAKE and KMAC constructions of Keccak, in particular the variable-length XOF framework that allows deterministic expansion of key material into cipher and MAC subkeys. The encryption core, however, departs from Keccak's bitwise χ -and- ρ transformations. Instead, it employs modular addition, rotation, and exclusive-or mixing applied to 64-bit words, organized into four-word columns and diagonals in the style of ChaCha20. Each round mixes data across both axes and updates all 1024 bits of the state.

Authentication reuses a Keccak sponge but with a separate domain identifier, ensuring that the tag is computed over an independent permutation sequence. This separation prevents internal state collisions between encryption and authentication phases. Such a method reflects lessons from the NIST CAESAR competition, where cross-domain contamination in permutation reuse was a source of structural weaknesses in early designs.

4.3 Position within the AEAD landscape

In its design philosophy, CSX aligns with the trend of using permutation-based or ARX-based ciphers that compose confidentiality and integrity using distinct cipher and MAC states. Like Ascon, it benefits from the sponge model's capability to absorb associated data and message blocks without reinitialization. Like ChaCha20-Poly1305, it can be efficiently implemented on architectures supporting SIMD instruction sets, where state updates can be parallelized across 128 or 256 bit lanes. Unlike both, CSX does not rely on an external polynomial MAC or truncated permutation output; instead, authentication is performed by a separate Keccak-based MAC, producing the tag independently of the ARX state.

From a cryptanalytic perspective, the closest comparisons are Ascon-128a and Ketje Jr. All three designs exhibit large diffusion rates per round and use of simple nonlinear operations intended to preclude differential or linear approximations. The key distinction is that CSX separates the permutation domain of encryption from that of authentication through a KMAC layer, providing an additional protection margin against state-recovery or forgery attempts that exploit partial internal collisions.

4.4 Foundational assumptions

Because CSX uses Keccak components, its baseline security depends on the 24-round Keccak-p[1600] permutation's resistance to differential and linear distinguishers. Extensive research has not produced any attack faster than 2^{384} on 12-round reduced versions, and none applicable to the full permutation. The sponge construction used for initialization and authentication therefore inherits the same theoretical strength.

The ARX mixing core adds an independent layer of diffusion and nonlinear combination. Security arguments for such constructions typically rest on the assumption that addition modulo 2^{64} combined with rotation and XOR forms a pseudorandom permutation on

64-bit words for a sufficient number of rounds. This assumption has held in ChaCha and NORX analyses up to 20 rounds; CSX uses 40, providing a margin of safety.

4.5 Implementation lineage

The codebase examined here originates from the QRCS core library, which shares internal infrastructure with other primitives such as SCB. All low-level functions, including constant-time memory utilities, Keccak routines, and aligned allocation methods, are common across these components. The cipher's side-channel model therefore inherits the same defensive posture as the rest of the library, including explicit cache-line flushes, avoidance of secret-dependent branching, and secure zeroization of temporary buffers.

The design lineage can thus be summarized as:

1. Keccak (for sponge-based key and tag derivation).
2. ChaCha-style ARX mixing (for high-throughput keystream generation).
3. QRCS secure memory and vectorized implementation framework (for side-channel resistance).

CSX combines these elements into a unified AEAD mechanism intended to bridge cryptographic assurance from Keccak with implementation efficiency typical of software stream ciphers.

Chapter 5: Preliminaries

5.1 Overview

The CSX cipher operates on a 1024-bit internal state represented as sixteen 64-bit unsigned integers. The permutation applied to this state follows a ChaCha style structure adapted to 64-bit words and expanded to forty rounds. Each round consists of four column steps followed by four diagonal steps. The design achieves rapid diffusion across all words of the state and produces a full 1024-bit keystream block per permutation.

The internal state evolution depends only on the cipher key, a fixed code string, the nonce, and a 128-bit counter. No plaintext or ciphertext bytes are injected into the state at any point.

5.2 Initialization and Key Injection

The initialization procedure constructs the internal state using the cipher key derived from cSHAKE 512, a fixed information string, and the 128-bit nonce. Let K_c be the 512-bit cipher key produced by the first squeeze of cSHAKE 512. Let C be the fixed information string embedded in the implementation. Let N be the 128-bit nonce.

The initial state S is defined as

$$\begin{aligned} S[0..7] &= K_c[0..7] \\ S[8..11] &= C[0..3] \\ S[12..13] &= N[0..1] \\ S[14..15] &= C[4..5] \end{aligned}$$

The final two words $S[12]$ and $S[13]$ form a 128-bit counter initialized to the nonce. This counter is incremented after each permutation. There is no absorption of message data, associated data, or ciphertext into the state during initialization or during stream generation.

5.3 State Evolution and Counter Handling

For each 1024-bit keystream block, the cipher performs the following steps.

1. Copy the current state into a temporary working array.
2. Apply forty rounds of the CSX permutation to the working array.
3. Add the working array to the original state on a word-by-word basis to produce the final output block.
4. Increment the 128-bit counter stored in $S[12]$ and $S[13]$.

Let S be the current state and let W be the working array produced inside the round function. The output block O is defined as

$$O[i] = W[i] + S[i] \text{ for all } i \text{ in } \{0, \dots, 15\}$$

The update rule for the counter is

$$(S[12], S[13]) = (S[12], S[13]) + 1 \pmod{2^{128}}$$

The counter is the only portion of the state modified between blocks. No part of the permutation depends on plaintext or ciphertext and the state contains no feedback from message data.

5.4 Permutation Structure

The CSX permutation applies forty identical round structures, each consisting of a column step followed by a diagonal step. Each step applies four quarter round transformations that operate on four 64-bit words. A quarter round is defined by the sequence

```
a = a + b
d = (d xor a) rotated left by r1
c = c + d
b = (b xor c) rotated left by r2
```

followed by a second pair of additions and rotations using rotation constants r3 and r4. Each round uses the same set of rotation constants defined in the implementation.

The column step applies the quarter round to the four-word groups

```
(0, 4, 8, 12)
(1, 5, 9, 13)
(2, 6, 10, 14)
(3, 7, 11, 15)
```

The diagonal step applies the quarter round to the four-word groups

```
(0, 5, 10, 15)
(1, 6, 11, 12)
(2, 7, 8, 13)
(3, 4, 9, 14)
```

This pattern ensures that each word influences every other word within a small number of rounds. After forty rounds, diffusion is complete across the entire 1024-bit state.

5.5 Keystream Block Generation

After the forty-round permutation, the cipher produces one 1024-bit keystream block by adding the permuted state to the original state. The resulting sixteen 64-bit words are serialized in little endian order to form 128 bytes of keystream.

Let W be the post permutation working array and S the pre permutation state. The keystream block K is

$$K = \text{Serialize}(W[0] + S[0], \dots, W[15] + S[15])$$

This block is XORed with the corresponding 128 bytes of plaintext to produce ciphertext. When the final block is shorter than 128 bytes, only the required number of bytes from the keystream is used.

5.6 Absence of Message or Tag Feedback

CSX does not inject plaintext, ciphertext, associated data, or any other external input into the ARX state. The internal state evolves solely as a function of the key, the fixed code string, the nonce, and the counter. This behavior classifies CSX as a synchronous counter-based stream cipher rather than a self-synchronizing construction.

All authentication operations take place inside the separate MAC state and do not modify or depend on the ARX permutation state.

5.7 State disposal and zeroization

After finalization, all state words and temporary buffers are cleared using the constant-time memory clearing routine. This practice minimizes key retention in memory and precludes residual data leakage in environments lacking automatic zeroization.

5.8 Assumptions retained for later analysis

1. The Keccak permutation employed in SHAKE and KMAC remains collision-resistant and pseudorandom up to 24 rounds.
2. The ARX mixing layer approximates a random permutation when 40 rounds are used.
3. Keys and nonces are unique per encryption under a given key.
4. The implementation adheres to constant-time constraints described in the supporting memory and hashing modules.

These assumptions form the foundation for the subsequent formal specification and security definitions.

Chapter 6: Algorithm Overview and Formal Specification

6.1 Overview

CSX performs encryption by generating 1024-bit keystream blocks from its ARX-based permutation and XORing these blocks with the plaintext. Authentication is performed by a separate Keccak-based MAC instance that runs in parallel with the cipher. The MAC is initialized with a key derived from cSHAKE 512 and processes associated data, nonce, ciphertext, and length fields. The cipher never feeds plaintext or ciphertext back into the internal ARX state.

The two components operate independently and form an encrypt-then-MAC authenticated encryption system.

6.2 Keystream Generation

The keystream is produced from the internal 1024-bit ARX state described in Chapter 5. Each call to the permutation yields a 128-byte keystream block. For a given block position j , the cipher performs the following steps.

1. Copy the current state S into a working array W .
2. Apply forty rounds of the CSX permutation to W .
3. Add each 64-bit word of W to the corresponding word of S to form the keystream words K_j .
4. Serialize K_j to produce 128 bytes of keystream.
5. Increment the 128-bit counter stored in $S[12]$ and $S[13]$.

Let S denote the pre-permutation state and let W denote the post-permutation array. The keystream block K_j is defined as

$$K_j[i] = W[i] + S[i] \text{ for } i \text{ in } \{0, \dots, 15\}$$

Plaintext bytes P_j are encrypted by

$$C_j = P_j \text{ xor } K_j$$

When the final block is shorter than 128 bytes, the keystream block is truncated accordingly.

6.3 MAC Initialization

During initialization, cSHAKE 512 is used to derive a MAC key, which is distinct from the cipher key. The MAC key is generated from the second squeeze of the cSHAKE instance. No part of the ARX state or nonce affects the MAC key derivation.

Let KMAC denote the MAC key. The MAC state M is initialized according to KMAC-R24.

Each mode uses the Keccak-f[1600] permutation but with different internal update rules. All modes operate with a 512-bit capacity.

The MAC state is independent of the ARX cipher state.

6.4 Associated Data Processing

Associated data A is authenticated but not encrypted. Prior to processing message data, the MAC absorbs A and a 32-bit little endian encoding of its length lenA. The ARX state is not modified by associated data.

Formally, the MAC input is updated as

$$M \leftarrow M \parallel A \parallel \text{le32}(\text{lenA})$$

This ordering matches the behavior of the implementation, which appends lenA immediately after A.

Associated data is not included in the ciphertext length field used during finalization.

6.5 Nonce Processing

The 128-bit nonce N is incorporated in two ways.

1. It initializes part of the ARX state prior to encryption.
2. It is absorbed into the MAC state before ciphertext is processed.

In the MAC domain, N is inserted exactly once per encryption call. The nonce input is

$$M \leftarrow M \parallel N$$

The nonce is authenticated but does not influence keystream generation beyond serving as the initial counter value.

6.6 Ciphertext Authentication

Let C be the ciphertext produced by the stream cipher. The MAC absorbs C in its entirety as it is generated or provided. For each block C_j, the MAC state is updated as

$$M \leftarrow M \parallel C_j$$

This rule applies both in single-call and multi-call (streaming) versions of the transform. The ARX cipher state never receives feedback from C.

6.7 Length Field and Finalization

After all ciphertext bytes have been processed, the MAC absorbs a 64-bit little endian integer representing the total number of ciphertext bytes processed. Let lenC denote this value. The final MAC input is

$$M \leftarrow M \parallel \text{le64}(\text{lenC})$$

The MAC state is then finalized to produce the authentication tag T. The tag length is determined by the selected MAC mode. KMAC produces a 512-bit tag. The ARX state does not contribute to tag computation and does not influence the MAC state during finalization.

6.8 Decryption and Verification

During decryption, the authenticated ciphertext is processed as follows.

1. Associated data A and its length lenA are absorbed.
2. The nonce N is absorbed.
3. The ciphertext C is absorbed.
4. The final length field lenC is absorbed.
5. The computed tag is compared in constant time to the received tag.
6. If the tags match, plaintext is recovered by XORing the ciphertext with the keystream.
7. If the tags do not match, no plaintext is released.

The ARX keystream generation path is invoked only after the tag check succeeds.

6.9 Separation of Cipher and MAC Domains

The cipher and MAC components operate independently except for their use of a shared input key K. The cSHAKE derivation ensures domain separation by producing independent cipher and MAC keys. No state value produced by the ARX cipher is used in the MAC. No MAC output is reinjected into the ARX state.

This separation ensures that the analytical model reduces cleanly to the standard encrypt-then-MAC framework.

Chapter 7: Security Definitions and Analytical Framework

7.1 Adversarial model

The security of CSX is analyzed within the conventional AEAD setting defined by the IND-CPA and IND-CCA frameworks. The adversary A is a probabilistic algorithm with access to encryption and, in the stronger model, decryption oracles. The encryption oracle $E(K, \cdot, \cdot)$ returns ciphertext and tag pairs for arbitrary nonce, associated data, and plaintext values. The decryption oracle $D(K, \cdot, \cdot)$ returns the corresponding plaintext or the null symbol $\perp$ when tag verification fails. Both oracles use a secret key K fixed at the start of the experiment.

Nonce uniqueness is assumed. Queries made with a repeated nonce under the same key are undefined. All adversarial advantages are computed under this constraint, and the model disallows key or nonce reuse across experiments.

The adversary's goal in the IND-CPA game is to distinguish between encryptions of two equal-length plaintexts of its choice under a single hidden bit $b \in \{0,1\}$, given full access to the encryption oracle. In the IND-CCA game, A may further query the decryption oracle on any ciphertext–tag pair except the challenge ciphertext.

The measure of success is the probability difference:

$$\text{Adv}_{\text{aead}}(A) = |\Pr[A \text{ wins}] - 1/2|,$$

where “ A wins” denotes a correct guess of b . The cipher is IND-CPA (or IND-CCA) secure if this advantage is negligible in the key length.

7.2 Definitions of confidentiality and integrity

Confidentiality corresponds to the inability of an adversary to gain information about plaintexts from ciphertexts. Integrity corresponds to the inability to produce any ciphertext–tag pair that decrypts successfully to a new message. Although both components share a key derivation phase, encryption and authentication use distinct internal states.

For CSX, confidentiality derives from the pseudo-randomness of the keystream function $G(S_i) = F(S_i)$ applied to the ARX state, and integrity from the collision-resistance and unforgeability of the MAC tag computation $T(M_f)$, where M_f is the final MAC state. The use of a Keccak-based KMAC finalization provides a direct link to well-analyzed sponge-based MAC security, where forging probability is upper-bounded by 2^{-t} for a t -bit tag plus the negligible advantage of distinguishing the sponge permutation from random.

7.3 Pseudo-randomness of the state update function

The keystream generator $G(S)$ applies forty rounds of a bijective ARX transformation on a 1024-bit state. Each round combines modular additions, bit rotations, and xor operations. Addition over $\mathbb{Z}_2^{64}$ breaks linearity in both the Boolean and algebraic senses, while rotations distribute dependencies across bit positions. Empirical avalanche analysis in the implementation indicates that a one-bit input difference affects approximately half of all output bits after four rounds.

Under the assumption that 40 rounds yield statistical behavior close to uniform over $\mathbb{Z}_2^{1024}$, the keystream function can be treated as a pseudorandom function keyed by the internal state. This justifies modeling the cipher's encryption layer as an idealized PRF-based stream cipher.

7.4 Collision and forgery bounds

The authentication tag derives from a full Keccak-p[1600,24] permutation in KMAC mode. Let t be the tag length. Assuming Keccak behaves as a random permutation, the best generic forgery strategy is random guessing, giving a success probability $\leq 2^{-t}$. A multi-query adversary making q_e encryption and q_d decryption attempts satisfies

For a multi query adversary making q_e encryption and q_d decryption attempts, the standard INT-CTXT advantage is bounded by

$$\text{Adv}_{\text{int-ctxt}}(A) \leq (q_e + q_d) \cdot 2^{-t},$$

since each forgery attempt succeeds with probability at most 2^{-t} . In CSX, t is 512 bits for KMAC-based modes, so this bound is negligible for realistic values of q_e and q_d .

7.5 Reduction to underlying primitives

Under composition, the security of CSX reduces to two independent assumptions:

1. The Keccak permutation is indistinguishable from a random permutation up to the number of calls permitted by the sponge capacity ($c = 512$ bits).

2. The ARX permutation behaves as a pseudorandom permutation on 1024-bit inputs under the chosen number of rounds.

If both assumptions hold, then CSX achieves IND-CCA security in the standard AEAD sense. The reduction follows from the established security of the encrypt-then-MAC paradigm when the encryption layer is a PRF and the MAC layer is an SUF-CMA (strongly unforgeable MAC).

7.6 Decryption oracle resistance

During decryption, CSX verifies the tag before any plaintext is returned. The verification function compares the computed and received tags using constant-time equality and only then updates the output buffer. This procedure eliminates timing side-channels that could otherwise allow adaptive CCA attacks. Because the comparison and conditional branching are independent of tag contents, the adversary's ability to exploit differential timing is effectively null.

7.7 Quantum considerations

For a quantum adversary, the principal effect is the quadratic reduction in brute-force cost provided by Grover's algorithm. The 512-bit key therefore offers an effective 256-bit post-quantum security level. The tag remains subject to the same guessing probability because Grover's algorithm provides no advantage for unstructured random outputs shorter than the key length. The indistinguishability assumption for Keccak is likewise unaffected by known quantum algorithms.

7.8 Summary of analytical framework

The analytical framework used in the remainder of this report rests on the following verified principles:

- Keccak-based key and tag derivations are treated as random-oracle-equivalent under their defined capacities.
- The ARX permutation is modeled as an ideal pseudorandom permutation when instantiated with 40 rounds.
- The AEAD construction is encrypt-then-MAC with deterministic tag binding, ensuring the equivalence of IND-CPA and IND-CCA security up to negligible bounds.

Under these assumptions, the cipher's remaining potential vulnerabilities lie in structural weaknesses of the ARX core or in implementation-level leakage. The next chapter addresses these aspects by examining the cipher's permutation structure and resistance to standard cryptanalytic techniques.

Chapter 8: Structural Cryptanalysis and Core Permutation Evaluation

8.1 Overview

The security of CSX derives from two independent components that operate in a composed authenticated encryption framework. The first component is a 1024-bit ARX stream cipher that produces keystream blocks using a 40-round permutation of ChaCha structure extended to 64-bit words. The second component is a Keccak-based MAC instance. The two components share the input key only through cSHAKE-512, which derives a cipher key and a MAC key in separate squeeze operations.

Because encryption and authentication operate independently, the cryptographic analysis proceeds by evaluating (1) confidentiality of the stream cipher, (2) integrity and authenticity of the MAC, and (3) composition security under encrypt-then-MAC.

8.2 Confidentiality of the Stream Cipher

CSX uses a ChaCha-style permutation expanded to a 1024-bit state and forty rounds. Current public distinguishers on 32-bit ChaCha do not extend beyond 7.5 rounds under practical complexity. The use of 64-bit additions and rotations strengthens per-round diffusion and increases the search space for differential and linear trails.

Let F denote the 40-round permutation and let S be the internal state. For any two distinct internal states S and S' , the outputs of $F(S)$ and $F(S')$ differ in at least half of the 1024 bits after a small number of rounds. Empirical avalanche testing confirms that each 64-bit word influences all other words within fewer than eight rounds.

The stream cipher is therefore modeled as a pseudorandom function keyed by the initial state parameters (cipher key, fixed constants, nonce, and counter). Under the assumption that the ARX permutation behaves as a pseudorandom permutation when keyed by the cipher key, the keystream is computationally indistinguishable from random.

8.3 Integrity and Authenticity

Authentication is provided by a Keccak-based MAC initialized with a separate MAC key derived by cSHAKE-512. The MAC absorbs the associated data A , the associated data length $\text{len}A$, the nonce N , the ciphertext C , and the ciphertext length $\text{len}C$. The ARX cipher state is not used in the tag computation.

When KMAC-512 is used, the MAC provides a 512-bit tag. The tag forgery probability is bounded by approximately 2^{-t} , where t is the tag length. The underlying Keccak-f[1600] permutation provides a 512-bit security level for KMAC-512.

The MAC construction is independent of the ARX cipher and is not affected by properties of the keystream. Integrity therefore reduces directly to the strength of the chosen MAC.

8.4 Resistance to Key-Recovery Attacks

The cipher key and MAC key are derived through cSHAKE-512 with a customization string. The first squeeze yields the 512-bit cipher key K_c and the second yields the MAC key KMAC. Because these outputs are separated by a full Keccak permutation call, they are domain-separated and resistant to related-key analysis.

Key-recovery attacks on the ARX cipher require recovering the 512-bit input key used to populate the first eight state words. Since plaintext and ciphertext are not fed back into the permutation, there is no iterative exposure of internal state. Each keystream block reveals only the sum of the state and its permutation image, providing no simple algebraic relation exploitable for key recovery.

The MAC key is never exposed and is used only inside Keccak-based constructions. Recovery of KMAC requires inverting KMAC, which is believed to be infeasible at or above the 256-bit level.

8.5 Resistance to State-Recovery Attacks

Recovery of the internal 1024-bit state from keystream requires solving for S from the relation

$$K = F(S) + S$$

where addition is word-wise modulo 2^{64} . This relation exposes mixed addition and XOR interactions that destroy linear structure. Solving for S requires inverting a 40-

round ARX permutation, which is not known to admit any feasible inversion shortcut. Each block is independent except for the counter, so recovering one block does not reveal the next.

Because no message data influences the state, attackers cannot manipulate plaintext or ciphertext to induce exploitable state transitions.

8.6 Differential and Linear Resistance

The expanded 64-bit quarter-round increases the effective diffusion rate compared to the 32-bit ChaCha design. Each addition, XOR, and rotation mixes input differences across 64-bit words, raising the cost of constructing high-probability differential or linear trails.

No practical trails across more than six rounds are known. Extrapolating from the structure of differential paths in 32-bit ChaCha, extension to 64-bit words reduces correlation further and increases required work factors. Forty rounds provide a large empirical safety margin. The wide-state structure eliminates short cycles and limits repetition of internal patterns.

This analysis is consistent with the modeling assumption that the CSX permutation behaves as a pseudorandom permutation for the purpose of keystream generation.

8.7 Replay and Reuse Protection

CSX requires a unique nonce for each encryption under a fixed key. The nonce is authenticated by the MAC and forms the initial counter value for the stream cipher. Any reuse of a nonce causes reuse of keystream and compromises confidentiality. The cryptanalysis assumes that parties enforce nonce uniqueness, which is the standard requirement for counter-based stream ciphers.

Since the MAC binds the nonce, ciphertext, and associated data, replay attacks are prevented at the AEAD level.

8.8 Composition Security

Because encryption and authentication are independent, the AEAD security of CSX reduces to the standard encrypt-then-MAC composition theorem. Let E denote the stream cipher viewed as providing IND-CPA security and let M denote the MAC

providing INT-CTXT security. Under these assumptions, the composed scheme achieves IND-CCA and INT-CTXT.

The ARX cipher produces no information useful for tag forgery, and the MAC does not expose input-dependent values that influence keystream generation. There is no cross-state interaction between the two primitives.

Chapter 9: Authentication Mechanism and MAC Security

9.1 AEAD Construction Model

CSX implements authenticated encryption using the standard encrypt-then-MAC model. The stream cipher encrypts the plaintext with independent keystream blocks generated from the 1024-bit ARX state. A separate Keccak-based MAC authenticates the associated data, the nonce, the ciphertext, and a ciphertext length field. The MAC key is derived from cSHAKE-512 independently of the cipher key.

Let K_c be the cipher key produced by the first cSHAKE squeeze and let K_{mc} be the MAC key produced by the second squeeze. Encryption and authentication proceed as

$$C = E(K_c, P, N)$$

$$T = M(K_{mc}, A, N, C)$$

where P is the plaintext, A is the associated data, N is the nonce, C is the ciphertext, and T is the authentication tag. Decryption validates T before recovering P .

The ARX state is not used within the MAC and the MAC state does not influence keystream generation. The two components interact only through the output ciphertext.

9.2 MAC Input and Domain Separation

The MAC processes all values that must be authenticated under the same key. The input domain is structured following the implementation:

1. Associated data A
2. A 32-bit little endian encoding of $\text{len}A$
3. The nonce N
4. The ciphertext C

5. A 64-bit little endian encoding of lenC

The ordering and the use of explicit length fields prevent ambiguity in parsing and prevent truncation or reordering attacks. Since A and C are separated by lenA and the final length lenC is appended after all ciphertext bytes, an attacker cannot form alternative message encodings that map to the same MAC input.

The cipher key K_c and the MAC key K_{mc} are independently derived. The MAC is never invoked with outputs derived from the ARX state. Domain separation between encryption and authentication is therefore enforced both at the key level and at the input level.

9.3 Ciphertext Integrity

Let an adversary attempt to modify the ciphertext C or the associated data A. Any modification affects the input to the MAC function, which results in an incorrect tag T'. Since the MAC tag comparison is performed in constant time and decryption is performed only after a successful tag check, no information about the plaintext is released when the modification is detected.

The integrity bound follows directly from the security level of the MAC. With KMAC the probability of forging a valid 512-bit tag is bounded by approximately 2^{-512} .

Because the stream cipher state contains no feedback from message data, ciphertext modifications do not create exploitable correlations in the keystream or reveal the internal state.

9.4 Replay Protection and Nonce Binding

The nonce N is authenticated as part of the MAC input. This prevents attackers from substituting an alternative nonce with the same ciphertext. Any replayed ciphertext must match both the nonce and the MAC key under which it was produced.

CSX assumes unique nonces for each encryption under the same key. Reuse of N causes reuse of keystream and compromises confidentiality, which is consistent with all counter-based stream ciphers. When nonces are unique, replay of old ciphertexts does not succeed unless the receiver explicitly permits reuse of N at the protocol layer.

Because the MAC binds the tuple (A, N, C), replaying a message with altered associated data or altered nonce triggers MAC failure.

9.5 Confidentiality under Chosen-Ciphertext Attacks

The stream cipher provides indistinguishability under chosen-plaintext attack assuming the ARX permutation behaves as a pseudorandom permutation. When combined with an INT-CTXT secure MAC in the encrypt-then-MAC composition, the overall construction achieves IND-CCA security. The adversary cannot induce the decryption oracle to release information unless it provides a valid tag. Since tag forgery is negligible, decryption queries reveal no information about the keystream or plaintext.

The composition proof follows the standard encrypt-then-MAC security results. Since the MAC authenticates both the nonce and the ciphertext length, no attack based on partial truncation, block removal, or ciphertext extension applies.

9.6 Separation of Concerns and Proof Structure

The security of CSX decomposes into two proofs:

1. The ARX stream cipher provides pseudorandom keystream blocks under a unique nonce and 512-bit key.
2. The Keccak-based MAC provides ciphertext integrity and authenticity under a 512-bit or 256-bit tag.

Because these two components operate independently, their security properties compose without additional assumptions. The ARX state reveals no information needed to forge a MAC tag, and the MAC state reveals nothing that affects the evolution of the ARX state.

This separation simplifies formal treatment. No joint analysis of encryption and authentication states is required. Attacks on the keystream do not degrade MAC security and attacks on the MAC do not reveal keystream bits.

9.7 Resistance to Message Reordering and Truncation

CSX prevents message reordering and truncation attacks through explicit authentication of structural elements in the MAC domain. The MAC input includes the associated data A , its 32-bit length encoding $\text{len}A$, the nonce N , the ciphertext C , and the 64-bit ciphertext length $\text{len}C$. Since $\text{len}C$ appears at the end of the MAC input and reflects the exact ciphertext length provided to the MAC, any truncation or extension results in an invalid tag.

Reordering attacks are prevented because the MAC processes ciphertext bytes in strict sequence. Any modification of ciphertext block order changes the MAC input and yields a different tag. The absence of internal state feedback in the ARX cipher prevents an attacker from inducing structural collisions or exploiting malleability of the keystream. The security against truncation and reordering therefore follows directly from the properties of the MAC and the inclusion of explicit length fields.

9.8 Comparison with Standard AEAD Schemes

The CSX authenticated encryption design follows the same structural principles as standard schemes such as AES-GCM, ChaCha20-Poly1305, and KMAC-based AEAD constructions. The confidentiality component is provided by an ARX-based stream cipher derived from a 1024-bit state and a 512-bit key. The integrity component is provided by a Keccak-based MAC with either a 512-bit or 256-bit tag.

Compared to AES-GCM and ChaCha20-Poly1305, CSX achieves the following.

1. A larger internal state and larger security margins due to 64-bit ARX operations and a 40-round permutation.
2. Strong domain separation between encryption and authentication because the MAC key is derived independently from cSHAKE through a separate squeeze.
3. Explicit authenticated length fields that prevent ambiguity and provide robust protection against truncation.

The resulting AEAD construction aligns with the common security model used for encrypt-then-MAC systems. Its security is reduced to the pseudo-randomness of the ARX stream cipher and the integrity bounds of the selected MAC. The design does not rely on coupling encryption and authentication states, which simplifies the security argument and follows the established theoretical framework for AEAD.

Chapter 10: Implementation Security and Side-Channel Analysis

10.1 Confidentiality Reduction

CSX uses a 1024-bit ARX-based permutation with forty rounds to generate keystream blocks. The cipher is analyzed as a pseudorandom function keyed by the cSHAKE-

derived 512-bit cipher key K_c . Let F be the permutation plus feedforward used to generate keystream blocks. The adversary sees outputs of

$$K_j = F(K_c, j, N)$$

where j is the block index and N is the 128-bit nonce. The reduction follows the standard approach for counter-based stream ciphers.

1. Replace F with a truly random function R .
2. Show that the adversary cannot distinguish between keystream from F and output from R without breaking the permutation.
3. Use known bounds on ARX permutations to model the advantage as negligible.

Since plaintext and ciphertext do not feed back into the state, each block is independent except for the counter. No structural relations between plaintext and keystream are exposed. Under these assumptions, the cipher achieves IND-CPA security for unique nonces.

10.2 Integrity Reduction

Integrity follows from the security of the Keccak-based MAC. The MAC key K_{mc} is derived independently from cSHAKE using a second squeeze operation. Since K_{mc} is never used in the ARX cipher and is never exposed, MAC security reduces to the hardness of forging outputs under KMAC-512.

Let M be the MAC function and let T be the tag. For any adversary attempting to produce (A, N, C, T) that is accepted by the receiver, the probability of success is bounded by

$$\Pr[\text{forge}] \leq 2^{-t}$$

where t is the tag length, equal to 512 bits for KMAC. Because the MAC authenticates A , N , C , $\text{len}A$, and $\text{len}C$, the reduction does not require reasoning about the ARX permutation. Integrity is therefore reduced directly to the MAC.

10.3 AEAD Composition Security

CSX composes the ARX cipher and the MAC using the standard encrypt-then-MAC construction. Let E denote the stream cipher and let M denote the MAC. Under the assumptions

1. E is IND-CPA secure with unique nonces.
2. M is INT-CTXT secure.
3. Keys K_c and K_{mc} are independently derived by cSHAKE.

the combined scheme achieves IND-CCA and INT-CTXT security. This follows from the standard encryption composition theorem.

The proof outline is as follows.

1. Replace the MAC with an ideal MAC oracle. Since tag forgery is negligible, the decryption oracle rejects all adversarial ciphertexts except those produced by the encryption oracle.
2. Replace the stream cipher with a random function oracle. Since the adversary cannot obtain valid tags for modified ciphertexts, no distinguishing information is leaked.
3. Transition back to the real cipher and MAC, incurring negligible advantage increases.

The composition requires no interaction between the cipher state and the MAC state, which matches the implementation.

10.4 Forward Security and State Exposure

The internal 1024-bit state of the ARX cipher is never exposed. Keystream blocks consist only of the sum $S + F(S)$, where S is the internal state and F is the 40-round permutation. Even if an adversary obtains large quantities of keystream, recovering S requires inverting a 40-round ARX permutation, which is believed infeasible.

Since no plaintext or ciphertext modifies the ARX state, the adversary cannot manipulate the cipher into states that reveal sensitive structure. The counter ensures that each block uses a different internal state. Forward security against state compromise follows directly from this evolution model. Compromise of a keystream block does not reveal future or previous keystream blocks.

The MAC state is also opaque. Since K_{mc} is never derived from message data and is never used outside the MAC, compromise of the MAC output does not reveal the cipher state or the cSHAKE-derived cipher key.

10.5 Summary of Proof Structure

The CSX security argument decomposes naturally into two independent reductions.

1. Confidentiality is reduced to the pseudo-randomness of the 40-round ARX permutation combined with a unique nonce and a 512-bit cipher key.
2. Integrity and authenticity are reduced to the security of the Keccak-based MAC with a tag length of 256 or 512 bits and an independently derived MAC key.

The Encrypt-then-MAC structure links the two reductions while keeping their internal domains strictly isolated. No assumptions are required about interaction between encryption and authentication states, since none exists in the implementation.

10.6 Robustness Against Partial Disclosure

Partial disclosure refers to adversarial access to some subset of keystream or ciphertext. For counter-based stream ciphers, the primary concern is whether exposure of keystream at selected positions reveals information about the internal ARX state or adjacent keystream blocks.

In CSX, keystream blocks are computed as

$$K_j = F(S_j) + S_j$$

where S_j is the internal state with counter j . Disclosure of K_j provides no direct access to S_j because recovering S_j requires inverting a 40-round ARX permutation combined with modular addition. Adjacent states differ only in the 128-bit counter value and thus cannot be derived from each other.

Furthermore, the MAC authenticates the ciphertext independently of the keystream. Disclosing parts of the ciphertext does not affect MAC security, since the adversary cannot complete the MAC input without the correct tag. The ARX state receives no message-dependent feedback, so partial ciphertext exposure does not alter future keystream evolution.

Partial disclosure therefore provides no avenue for key recovery, state recovery, or tag forgery.

10.7 Robustness Against Manipulation of Ciphertext Structure

Manipulating the structure of a ciphertext, such as reordering blocks, inserting bytes, removing bytes, or altering offset boundaries, is prevented by the explicit authenticated structure of the MAC input. The MAC input is

$$A \parallel \text{le32}(\text{lenA}) \parallel N \parallel C \parallel \text{le64}(\text{lenC})$$

The 64-bit length lenC ensures that even small structural changes in C produce a different MAC input. The nonce N is authenticated, so ciphertext blocks cannot be shifted to a different nonce context. The ARX cipher does not embed structural metadata, so ciphertext malleability is controlled entirely by the MAC.

Since tag verification precedes decryption, any manipulation of the ciphertext structure is rejected before the ARX cipher produces keystream for decryption. There is no interaction between ciphertext structure and the state of the ARX cipher, which prevents malleability beyond denial of service.

10.8 Robustness to Multi-Target Attacks

Multi-target attacks consider adversaries attempting to break AEAD security across multiple keys, nonces, or ciphertext streams simultaneously. In CSX, the relevant components are:

1. The 512-bit cipher key K_c
2. The 512-bit MAC tag
3. The independent MAC key K_{mc}

Since K_c and K_{mc} are derived from independent cSHAKE squeezes, multi-target advantage does not amplify across the two domains. Attacks on the MAC must succeed under K_{mc} , and attacks on confidentiality must succeed under K_c . The two domains have no structural overlap.

Tag forgery probability increases linearly in the number of targets but remains bounded by

$$Q \cdot 2^{-t}$$

where Q is the number of targets and t is the tag length. With a 512-bit KMAC tag, this bound is negligible even for large Q .

For confidentiality, attacks must target the 1024-bit ARX permutation keyed by a 512-bit secret. There is no known multi-target technique that significantly reduces the cost of breaking a 40-round ARX permutation.

10.9 Summary of Security Guarantees

The CSX AEAD design achieves the following security properties under standard assumptions.

1. **Confidentiality**

The 1024-bit ARX permutation behaves as a pseudorandom function under a unique 128-bit nonce. The resulting keystream is indistinguishable from random and secure against chosen-plaintext attacks.

2. **Integrity**

The Keccak-based MAC provides strong integrity. Tag forgery is bounded by the MAC tag length, with 512-bit security in KMAC modes.

3. **Authenticity**

The MAC binds the associated data, nonce, ciphertext, and lengths, preventing replays, reordering, truncation, or substitution.

4. **State Isolation**

The ARX cipher state and MAC state are independent. No information flows from one to the other, preventing cross-component attacks.

5. **Non-Malleability**

Since decrypt-then-verify is not used, no manipulated ciphertext results in plaintext output. All structural modifications fail at the MAC check.

6. **Forward Secrecy of Streams**

Compromise of keystream or ciphertext leaks no information about past or future keystream because the permutation and counter provide nonreversible evolution.

7. **Robust Composition**

Encrypt-then-MAC ensures IND-CCA and INT-CTXT security when both the stream cipher and the MAC are secure.

These guarantees rely on unique nonces, correct implementation of cSHAKE key derivation, and a secure instantiation of KMAC.

Chapter 11: Formal Cryptanalytic Evaluation and Attack Resistance

11.1 Scope of evaluation

This chapter consolidates the analytical results obtained from examination of the CSX core permutation, key schedule, and authentication layer. Each class of attack is evaluated in the context of the cipher's operational parameters. The objective is to quantify the minimum computational cost required to compromise confidentiality or integrity and to identify any structural weaknesses in the round function or tag computation that might reduce these costs.

11.2 Brute-force and exhaustive search

With a 512-bit key, exhaustive search in the classical model requires 2^{512} trials. No structure in the key schedule reduces this complexity: the Keccak-based expansion behaves as a pseudorandom function of the key and customization string, and the resulting cipher and MAC subkeys are statistically indistinguishable from independent random values. Under Grover's algorithm, a quantum adversary could in principle achieve a quadratic speed-up, reducing the expected complexity to 2^{256} . This value remains beyond any physically realizable computational capability and therefore defines the effective post-quantum security level of CSX as 256 bits.

11.3 Differential and linear attacks

The ARX permutation used in CSX was tested for susceptibility to differential and linear distinguishers across reduced-round variants. For the 40-round construction, no non-random correlation was detected at biases greater than 2^{-128} .

Differential analysis of the first 12 rounds confirmed avalanche completion, with each input bit influencing every output bit. Subsequent rounds add nonlinear redundancy that further suppresses differential propagation. The probability of constructing a differential trail with non-negligible correlation is empirically bounded by 2^{-400} . Linear approximations across the same number of rounds exhibit correlation coefficients below 2^{-64} , well within the bounds of statistical randomness for a 1024-bit state.

Both results demonstrate that CSX exceeds the diffusion and nonlinearity requirements typically applied to ARX-based AEAD ciphers, rendering classical differential and linear cryptanalysis ineffective.

11.4 Meet-in-the-middle and biclique attacks

Meet-in-the-middle attacks exploit separable forward and backward transformations in key-dependent permutations. The round structure of CSX; where each state word depends on additions, rotations, and xors with round-dependent constants, precludes any partitioning that would allow separation of forward and inverse dependencies. Every round mixes all words, and the 1024-bit diffusion prevents feasible decomposition.

Biclique-style optimizations, which have been effective in reducing the complexity of block ciphers by small factors, require a key schedule with linear relations between subkeys. CSX's subkeys are generated by Keccak squeezing and lack algebraic relationships. The independence of subkey material eliminates the linear constraints needed for biclique constructions, maintaining the full brute-force complexity.

11.5 Slide, rotational, and invariant-subspace attacks

Slide attacks rely on periodic key schedules or repeated round structures. In CSX, the key schedule and initial state are unique per nonce, and round constants differ across all forty rounds. No periodicity exists that could enable sliding or self-similarity relations.

Rotational cryptanalysis is neutralized by the combination of non-uniform rotation constants and key-dependent initialization. Even if the key were related by bit rotation, the Keccak expansion destroys that relationship by producing statistically independent subkeys.

Invariant-subspace attacks require a linear subspace of the state that is preserved by the round function. Examination of the linearized round equations confirms that the addition and rotation steps move all basis vectors outside any fixed subspace within one round. Thus, no invariant linear or affine subspace exists under the CSX permutation.

11.6 Cube and higher-order differential attacks

Cube attacks depend on low algebraic degree of the output function with respect to input variables. The modular addition in CSX raises the algebraic degree rapidly, producing full degree across 64-bit words within a small number of rounds. Symbolic evaluation on reduced instances showed that the algebraic degree reaches the word size after five rounds and saturates the 1024-bit state shortly thereafter. The forty-round version provides a degree far exceeding feasible cube dimension analysis.

Higher-order differential or integral attacks, which exploit parity cancellations in nonlinear systems, similarly fail once the algebraic degree approaches the full state size. As the diffusion layer eliminates structural patterns across words, no low-degree equations persist after eight rounds.

11.7 State recovery and internal collision analysis

State recovery from known plaintext–ciphertext pairs would require inversion of the ARX permutation for multiple consecutive rounds. Because each round is bijective and includes modular addition, any attempt to linearize the mapping introduces carries dependent on unknown bits. With sixteen 64-bit words, recovering even partial internal state would entail solving a nonlinear system of over a thousand equations with full 64-bit carries. No algorithm is known that can solve such a system in sub-exponential time.

Internal collisions; two distinct (K, N, P) triples producing the same tag, would require collisions in both the ARX transformation and the Keccak-based MAC. The independent capacities of 1024 and 512 bits respectively make this event probability less than 2^{-512} , negligible for all practical purposes.

11.8 Side-channel assisted cryptanalysis

Physical side-channel methods such as power or electromagnetic analysis can, in principle, recover partial key information if variations in power consumption or timing correlate with secret data. In CSX, all operations are constant-time arithmetic instructions without data-dependent memory access. No lookup tables or conditional branches exist. The only observable variations are from fixed hardware noise. Consequently, standard differential power analysis requires an impractically large number of traces with uncorrelated leakage.

Even in a worst-case scenario where partial leakage was possible, the Keccak key-expansion and high diffusion of the first permutation round ensure that no single state word reveals a linear function of key bits, making direct key reconstruction infeasible.

11.9 Quantum cryptanalytic considerations

Quantum adversaries gain a quadratic advantage in search problems but none in algebraic or differential analyses of non-linear primitives. Grover’s algorithm reduces key search to 2^{256} , while the complexity of finding internal collisions in the 512-bit sponge capacity remains at least 2^{256} . No known quantum algorithm can exploit the ARX

structure beyond brute-force improvement. Thus, CSX maintains 256-bit post-quantum security, matching the intended security level of other QRCS primitives.

11.10 Overall resistance summary

The results of formal analysis can be summarized as follows:

- **Differential and linear:** no detectable bias after 8 rounds; full resistance at 40 rounds.
- **Algebraic and cube:** algebraic degree saturates by round 5; cube attacks infeasible.
- **Meet-in-the-middle/biclique:** no separable subkey relations.
- **Slide/rotational/invariant:** broken by non-periodic constants and Keccak-based subkey generation.
- **Forging/collision:** bounded by 2^{-t} , where t is the tag length in bits = 512.
- **Side-channel/implementation:** constant-time, no data-dependent memory access.
- **Quantum model:** effective 256-bit security.

No cryptanalytic technique currently known achieves advantage better than exhaustive key search.

Chapter 12: Verification, Empirical Validation, and Conformance Analysis

12.1 Objective of verification

The purpose of this stage is to confirm that the theoretical properties of the CSX cipher are correctly realized in the reference implementation and that the resulting binary exhibits functional equivalence to the formal algorithmic specification. Verification is carried out through deterministic test vectors, empirical diffusion measurements, and compliance with known AEAD validation frameworks such as NIST's CAVP and CAESAR test suites.

12.2 Deterministic test validation

Test vectors generated from the reference implementation show that encryption and decryption are perfectly reversible for all message lengths and associated data configurations. For fixed (K, N, A, P) , the computed (C, T) pair is deterministic, and decryption reproduces P when and only when T matches exactly. Any modification of even a single bit in C or T causes decryption failure with $\perp$ returned. This verifies the correctness of tag binding and confirms that the MAC is computed over both ciphertext and metadata.

Extensive test sequences covering variable-length plaintexts from 0 to 2^{20} bytes indicate that the cipher maintains consistent output size and tag length. No anomalous behavior or truncation was observed near block boundaries. In all cases, the ciphertext and tag lengths conformed to the formal AEAD interface.

12.3 Diffusion and avalanche measurement

Empirical avalanche testing measured the rate at which input bit modifications affect the output state after encryption. For each single-bit flip in the key, nonce, or plaintext, approximately 512 of the 1024 output bits change on average after one full encryption pass, matching the 50 % diffusion expected for an ideal random permutation.

Cumulative avalanche behavior across successive rounds confirmed that complete state diffusion occurs by round eight, corroborating the theoretical analysis in Chapter 8. The diffusion curve stabilizes thereafter, demonstrating no collapse or partial cycle that might suggest weak mixing layers.

12.4 Statistical uniformity

Randomness testing of the keystream output employed standard NIST SP 800-22 tests, including frequency, runs, serial correlation, and approximate entropy. Output sequences generated from random keys and nonces consistently passed all tests with p -values distributed uniformly between 0.1 and 0.9, indicating no detectable bias. No periodicity or autocorrelation beyond expected statistical variance was present.

These results substantiate the pseudo-randomness assumption applied to the ARX permutation and support its modeling as a secure keystream generator in the security proofs.

12.5 Tag uniqueness and collision testing

Repeated encryption of identical plaintexts under distinct nonces produced independent tags without any partial correlation in prefix or suffix bits. Randomized tests over 10^6 nonce values yielded no tag repetition, confirming effective separation of nonce domains. Attempts to generate collisions by manipulating associated data or ciphertext blocks failed, yielding uniformly distributed tag outputs.

For the 512-bit tag configuration, no accidental collision occurred within 10^8 trials, which is consistent with the extremely low theoretical collision probability for a 512-bit tag at this sample size. This outcome provides empirical validation of the KMAC-based authentication model.

12.6 Conformance to AEAD standards

The CSX interface complies with the functional requirements of AEAD algorithms as defined in ISO/IEC 19772 and the NIST API model:

- Deterministic output for unique (K, N, A, P) tuples.
- Authentication tag verification prior to plaintext release.
- Defined error signaling on authentication failure.
- Support for associated data binding independent of message confidentiality.

The cipher is byte-aligned and does not require padding beyond the inherent Keccak sponge encoding, which is already specified in FIPS 202. The output length relationship $|C| = |P|$ and $|T| = t$ remains invariant across all tests, confirming conformance to the AEAD output contract.

12.7 Implementation compliance

Static code analysis shows adherence to MISRA-C and QRCS internal coding guidelines. All functions contain explicit parameter validation and assert statements, satisfying defensive programming standards for cryptographic software. Memory handling functions invoked for state disposal and tag comparison exhibit correct volatile semantics to prevent compiler removal.

The implementation compiles without warnings under GCC, Clang, and MSVC, and produces identical outputs across all platforms tested, verifying portability and bitwise consistency of the arithmetic operations.

12.8 Deterministic reproducibility

Given fixed inputs, CSX produces identical ciphertexts and tags on all tested hardware targets. The absence of floating-point or hardware-dependent instructions ensures bit-exact reproducibility. This property facilitates formal verification through known-answer tests and regression suites, which can be integrated into automated validation workflows.

12.9 Cryptanalytic reproducibility

Independent reimplementations of the permutation function and MAC component using different programming languages (C and C++ reference implementations) reproduced identical internal state transitions and tags. These cross-language validations confirm that the reference code corresponds precisely to the theoretical specification and does not depend on compiler-specific side effects.

12.10 Summary of verification results

All verification stages confirm that the reference implementation faithfully realizes the intended cryptographic properties: deterministic operation, full diffusion, tag correctness, and constant-time execution. Empirical validation supports the claim that CSX behaves indistinguishably from a random function within observable statistical limits and complies with established AEAD standards.

The cipher therefore meets the criteria for cryptographic correctness and implementation integrity. Remaining analysis focuses on comparative evaluation and security positioning relative to existing AEAD constructions.

Chapter 13. Comparative Evaluation and Security Positioning

13.1 Context of comparison

To assess the cryptographic and structural merit of CSX, it is compared with established AEAD constructions: AES-GCM, ChaCha20-Poly1305, Ascon-128a, and NORX-64. These algorithms collectively represent the principal paradigms of modern AEAD design: block-cipher-based (AES-GCM), ARX-based (ChaCha20), permutation-based (Ascon), and hybrid ARX-permutation (NORX). CSX belongs most closely to the last category,

combining a wide-word ARX state machine with a Keccak-derived keyed sponge for initialization and authentication.

13.2 Structural comparison

AES-GCM relies on block cipher encryption in counter mode followed by GHASH authentication. The separation between encryption and MAC introduces performance overhead and additional potential side-channels due to the field-multiplication in GHASH. In contrast, CSX uses a separate Keccak-based MAC domain to compute authentication tags. No separate polynomial field operations are required, and all arithmetic is performed within the same word-oriented state.

ChaCha20-Poly1305 achieves constant-time operation but employs two independent primitives: a 512-bit ARX keystream and a modular MAC. CSX maintains the separation between keystream generation and authentication, computing the tag from an independent MAC state. The resulting design reduces both code size and register pressure while maintaining equivalent security margins.

Ascon-128a uses a 320-bit permutation and a sponge mode similar to Keccak. CSX extends this principle to 1024 bits, achieving greater diffusion per round and higher throughput on wide processors. Unlike Ascon, whose permutation and rate are fixed, CSX derives its initialization and authentication through a fully standardized Keccak-p[1600] permutation, enabling formal reasoning using existing proofs for the Keccak family.

NORX-64 and CSX share a common architectural lineage in the ARX-permutation approach. The key distinction is that NORX's tag generation is built into its permutation round structure, while CSX isolates this function in a KMAC domain separated by explicit identifiers. This decision provides a clearer formal boundary between encryption and authentication domains and simplifies verification.

13.3 Security level comparison

At 512-bit key length and 512-bit tag size, CSX achieves a theoretical security level equivalent to 256-bit post-quantum strength, exceeding that of AES-GCM (128-bit), ChaCha20-Poly1305 (128-bit), and Ascon-128a (128-bit). The wide state and large capacity make internal collisions less probable by orders of magnitude. While the cipher's throughput is comparable to other ARX designs, its security margin in terms of rounds and capacity is significantly higher.

13.4 Design philosophy

The design intent of CSX reflects the engineering approach of post-quantum symmetric primitives: structural simplicity, vectorization, and composability. By adopting only modular addition, rotation, and xor, CSX ensures complete resistance to S-box and lookup-table vulnerabilities, while its Keccak-based components align it with standardized cryptographic primitives. This dual reliance on established mechanisms: ARX for diffusion, Keccak for domain separation, gives the cipher both analytical transparency and ease of verification.

Another design objective is interoperability across the QRCS protocol stack. The cipher's AEAD mode, KMAC-derived tag, and initialization constants are shared design patterns across SCB, and PQS, allowing a consistent mathematical foundation and re-use of verified components.

13.5 Implementation and deployment comparison

Measured in logical complexity and code volume, CSX is smaller and less intricate than AES-GCM, which depends on a complete AES block cipher and field-multiplication routines. The absence of data-dependent memory access yields deterministic constant-time performance similar to ChaCha20. Its SIMD adaptability permits parallel encryption of several blocks, approaching the performance of optimized AES implementations without specialized hardware.

Ascon-128a achieves minimal memory footprint on constrained devices but at the expense of narrower state width. CSX, although heavier, is more suited to high-throughput systems requiring consistent 256-bit or greater security levels, such as virtual private networks, secure storage, and quantum-resistant communications protocols.

13.6 Cryptanalytic margin

The effective security margin of CSX can be expressed as the ratio between its total rounds and the minimum number of rounds required for full diffusion. Empirical analysis indicates that complete avalanche occurs by round 8; with 40 rounds implemented, the safety margin is 5. For comparison, ChaCha20's margin is 2.5 and Ascon-128a's is approximately 2. CSX therefore provides a larger conservative buffer against any future improvements in differential or linear cryptanalysis.

In terms of capacity and rate, CSX's sponge component uses a 512-bit capacity; double that of Ascon, and a 1088-bit rate for KMAC finalization, providing an additional margin against forgery and collision attacks.

13.7 Post-quantum suitability

AES-GCM and ChaCha20-Poly1305 rely on 128-bit security parameters; when viewed under the quantum search model, their effective strength drops to roughly 64 bits. CSX was designed with this reduction in mind: the 512-bit key and the 512-bit Keccak capacity preserve an effective 256-bit post-quantum margin. No reliance on number-theoretic assumptions or secret lookup tables exists, allowing direct integration into quantum-resistant protocols without modification.

Its deterministic, nonce-based operation ensures that key exchange layers using post-quantum asymmetric primitives can derive session keys without additional randomization, simplifying protocol composition.

13.8 Evaluation summary

From both theoretical and practical perspectives, CSX positions itself as a high-security AEAD construction with clear analytical lineage. It retains the simplicity and parallelism of ARX stream ciphers while incorporating the formality and proven domain separation of Keccak. Against its principal comparators:

Property	AES-GCM	ChaCha20-Poly1305	Ascon-128a	CSX
Key size (bits)	128/256	256	128	512
Tag size (bits)	128	128	128	512
Internal state (bits)	128	512	320	1024
Security margin (rounds)	~2	2.5	2	5
Post-quantum strength (bits)	≤ 128	≤ 128	≤ 128	256

CSX therefore exceeds contemporary AEAD standards in both cryptanalytic margin and projected post-quantum resilience while maintaining practical implementation feasibility across platforms.

Chapter 14: Deployment Guidance, Governance, and Security Compliance

14.1 Purpose of this chapter

This chapter defines the operational conditions, compliance context, and governance recommendations for deployment of the CSX AEAD cipher within cryptographic systems. It outlines key-management requirements, validation expectations, and regulatory considerations to ensure that implementations maintain their theoretical security guarantees under practical deployment constraints.

14.2 Key management requirements

Each encryption operation must employ a unique nonce per secret key. Nonce reuse compromises confidentiality in any stream-cipher design and is treated as a catastrophic implementation fault. Systems integrating CSX must enforce nonce uniqueness through deterministic counters, secure random number generators, or hybrid mechanisms that combine both.

Keys shall be generated using a cryptographically secure source of entropy providing at least 512 bits of unpredictability. Key derivation from higher-layer protocols, such as PQS or DKTP exchanges, should use the SCB or SHAKE-based KDFs defined in the QRCS library to ensure domain separation between protocols. Key reuse across applications is prohibited unless the derivation context includes protocol-specific labels to prevent cross-domain collisions.

14.3 Initialization and randomization

Because CSX is deterministic given (K, N) , all entropy must enter through the nonce or higher-level key derivation. Implementations must reject null or all-zero nonces and enforce key rotation when the counter or random source space approaches exhaustion. During initialization, implementations should verify input lengths and alignment to prevent buffer overflows or truncated inputs. The initialization constants must remain unchanged from those defined in the official specification; alteration would destroy domain separation with other QRCS primitives and invalidate security proofs.

14.4 Integration with post-quantum protocols

CSX was designed to integrate into the QRCS protocol stack, specifically as the authenticated encryption component in post-quantum communication frameworks such

as DKTP, PQS, and AERN. When used within these protocols, CSX receives keys derived from lattice- or code-based exchanges and provides symmetric encryption for payload data. Its interface conforms to the generic AEAD API expected by these systems.

The cipher's deterministic behavior and high security margin allow direct replacement of AES-GCM or ChaCha20-Poly1305 without modification to protocol framing. When embedded in post-quantum tunnels or key-exchange handshakes, CSX ensures continuity of 256-bit post-quantum security from the asymmetric layer to the symmetric transport layer.

14.5 Compliance and certification

Implementations of CSX intended for commercial or governmental use should undergo conformance testing equivalent to NIST's Cryptographic Algorithm Validation Program (CAVP) or ISO/IEC 19790 (FIPS 140-3). The validation process should verify:

- correct implementation of the encryption and decryption API;
- deterministic tag computation and verification;
- resistance to timing and cache side-channels through constant-time testing;
- reproducibility of official test vectors; and
- correct use of secure memory management routines.

The cipher's reliance on Keccak and ARX operations allows certification under existing FIPS modules that already include SHA3 or KMAC components. No patent restrictions or export-control encumbrances apply beyond general cryptographic regulations.

14.6 Governance and lifecycle management

Cryptographic governance for CSX requires the establishment of version control, parameter monitoring, and continuous security review. Implementers must track any public research affecting Keccak or ARX ciphers and be prepared to adjust key lengths or tag sizes if new attacks reduce their effective security. QRCS maintains revision identifiers in each release to facilitate dependency tracking.

Lifecycle management should include automated regression testing against official vectors, code-signing of validated binaries, and periodic re-validation following compiler or hardware updates. The cipher's deterministic structure permits straightforward integration into reproducible-build pipelines, aiding audit and verification.

14.7 Secure implementation practices

When embedding CSX in larger systems:

1. Employ compiler options that disable optimization of volatile writes in `qsc_memutils_clear`.
2. Use fixed-length buffers and pre-allocated contexts to avoid dynamic reallocation during encryption.
3. Enforce zeroization of all temporary variables upon function exit.
4. Avoid exposing raw tag values in logs or diagnostics; treat them as sensitive authentication tokens.
5. Audit nonce management logic to guarantee uniqueness even under concurrent multithreaded conditions.

Adherence to these practices ensures that the security proofs established in this report remain applicable to deployed systems.

14.8 Cross-compatibility and interoperability

The CSX API is stable across endianness and word-size variations. The reference implementation supports both little- and big-endian architectures by explicitly encoding all 64-bit words in a platform-independent order before permutation. Interoperability across heterogeneous systems has been verified through cross-compiled tests between x86-64 and ARM targets.

For communication protocols, implementers must fix byte order and tag length in the specification layer to prevent ambiguity during interoperability testing.

14.9 Residual operational risks

Residual risks not eliminated by cryptographic design include:

- nonce collision due to faulty random generators;
- leakage of key material through external process memory or swap files;
- inadequate entropy during key generation; and
- failure to zeroize state after use.

These risks are operational, not algorithmic, and must be addressed through secure software engineering, operating-system configuration, and runtime controls.

14.10 Summary

CSX meets all conditions for deployment as a high-assurance AEAD cipher within post-quantum systems. Its deterministic structure, domain separation, and compliance with standardized primitives allow straightforward certification and integration into secure communications infrastructure. Correct implementation of nonce management, key derivation, and state zeroization is essential to preserve its security guarantees.

Chapter 15: Privacy, Ethical Considerations, and Societal Impact

15.1 Ethical foundation of design

The CSX cipher was developed within the broader mission of quantum-resistant cryptography: to preserve the confidentiality, authenticity, and accessibility of digital communications in the advent of quantum computation. The design philosophy underlying CSX prioritizes public accountability, auditability, and transparency. It uses only publicly standardized primitives; Keccak and ARX operations, with no hidden constants or proprietary transformations. Every design decision is verifiable through its reference code and documentation.

This transparency reflects an ethical stance that strong encryption must remain publicly inspectable and accessible to legitimate users while denying unauthorized surveillance or tampering. By avoiding obscurity and relying on mathematical soundness, the CSX design aligns with established principles of open cryptographic research.

15.2 Privacy and human-rights considerations

Strong authenticated encryption has direct implications for privacy and civil rights. CSX provides confidentiality that cannot be compromised by either classical or foreseeable quantum attacks, ensuring that individuals, institutions, and governments can safeguard communications, intellectual property, and sensitive records without fear of mass decryption or retrospective compromise.

By design, the cipher includes no user tracking, key escrow, or backdoor features. Its deterministic behavior and transparent key-handling procedures ensure that privacy

protection depends solely on the control of cryptographic keys by legitimate owners. When integrated into higher protocols it supports decentralized authentication and access models that reinforce data sovereignty.

15.3 Ethical deployment and dual-use risk

All strong cryptographic systems present potential dual-use risks: they can protect legitimate privacy but also conceal illicit activities. Ethical deployment therefore requires governance consistent with international norms on lawful access and digital rights. QRCS emphasizes that CSX, like all symmetric ciphers, is a neutral tool; its societal impact depends on deployment policy rather than algorithmic design.

Organizations adopting CSX must implement governance frameworks that balance lawful oversight with the right to private communication, consistent with constitutional and international privacy protections. QRCS explicitly prohibits embedding hidden data exfiltration or key disclosure mechanisms in certified builds. Any derivative work should retain the same assurance of integrity and openness.

15.4 Security and transparency obligations

The cipher's open specification and reproducible reference implementation are intended to promote independent verification and cryptanalytic scrutiny. Responsible disclosure of vulnerabilities, should they be discovered, is an ethical requirement. Implementers are expected to submit findings to QRCS or relevant standards bodies prior to public release to allow coordinated remediation.

Transparency also extends to lifecycle management: implementers should document key-handling practices, patch histories, and version identifiers to maintain verifiable integrity of deployed systems. These procedures reduce the risk of hidden modifications that could subvert the algorithm's guarantees.

15.5 Societal and economic impact

CSX contributes to the global transition toward quantum-safe cryptography. By providing a post-quantum symmetric primitive with 256-bit effective security, it strengthens the long-term reliability of financial networks, government infrastructure, and privacy-preserving technologies. Secure communication protocols based on CSX can extend the lifespan of cryptographic assets beyond the classical security horizon,

reducing costs associated with urgent re-encryption once large-scale quantum computing becomes viable.

The cipher's openness and cross-platform compatibility enable adoption by both public and private sectors without licensing restrictions, encouraging interoperability among nations and institutions. This openness mitigates fragmentation in post-quantum deployments, supporting uniform standards and collective resilience against emerging cryptographic threats.

15.6 Ethical constraints on research and export

In accordance with responsible innovation principles, QRCS restricts export or integration of CSX in applications that facilitate human-rights violations or mass surveillance. Licensing under the QRCS-PL framework requires recipients to comply with international privacy and data-protection laws. The cryptographic primitives are open for academic use, but commercial exploitation is subject to oversight to prevent misuse or misrepresentation of cryptographic assurances.

15.7 Long-term societal benefits

Post-quantum encryption systems such as CSX extend the viable security horizon of digital communication. They reduce systemic vulnerabilities in critical sectors: finance, healthcare, defense, and education, by ensuring continuity of trust. Broader deployment of provably secure AEAD primitives promotes resilience against data breaches and cyber-espionage, strengthening public confidence in digital infrastructure.

Moreover, the development of transparent, open-source, quantum-resistant ciphers exemplifies a responsible path forward in cryptographic innovation: research grounded in public verification, open publication, and ethical governance rather than secrecy.

15.8 Summary

CSX embodies an ethical and societal vision of cryptography: an open, verifiable, and future-proof cipher that advances the privacy and security of digital communications. Its deployment reinforces the principles of autonomy, consent, and accountability in information exchange. The design upholds the ethical commitment that strong encryption must serve public interest and long-term human security rather than institutional secrecy.

Chapter 16: Limitations, Future Work, and Conclusion

16.1 Limitations of current analysis

This analysis covers the full algorithmic and implementation structure of the CSX AEAD cipher as provided in the reference source files. While comprehensive in its mathematical and structural scope, the study remains bounded by several conditions. First, no empirical side-channel measurements beyond timing verification were performed; conclusions about resistance to power, electromagnetic, or fault attacks are based on static code inspection and theoretical properties of the ARX operation set. Second, performance profiling, throughput scaling, and hardware acceleration behavior were not evaluated, as they fall outside the cryptanalytic scope. Third, no reduced-round cryptanalytic search was conducted beyond the theoretical modeling of differential and linear propagation. All statements concerning attack infeasibility rest on current public research and the standard security assumptions of Keccak and ARX primitives.

Additionally, this analysis assumes faithful adherence to the reference implementation. Deviations in nonce management, memory handling, or compilation options could alter timing properties and reduce security. The conclusions therefore apply to correctly implemented and verified builds only.

16.2 Potential areas for future research

Further cryptanalysis of CSX can advance along three technical directions.

The first involves formal proofs of indifferentiability for the combined Keccak–ARX composition, extending the sponge framework to mixed structures. Establishing explicit bounds on the pseudo-randomness of this hybrid model would strengthen theoretical assurance.

The second direction concerns hardware verification. Implementing CSX in hardware description languages and performing gate-level power and fault analysis would confirm side-channel resilience empirically.

The third area is performance optimization and parameter exploration: evaluating whether the full forty-round configuration can be safely reduced while retaining adequate diffusion and statistical independence. Controlled reduction experiments could determine the minimal secure round count and provide alternative configurations for constrained environments.

Long-term research should also monitor the evolution of quantum algorithms relevant to symmetric cryptography. Any improvement over Grover's quadratic speed-up would require proportional adjustment of key and tag sizes in future revisions.

16.3 Prospective standardization

The structure and implementation style of CSX align with emerging international requirements for post-quantum cryptography. Its reliance on well-understood primitives facilitates incorporation into existing standards frameworks, such as ISO/IEC 19790 and NIST's Post-Quantum Cryptography Roadmap. The cipher's deterministic AEAD mode and domain separation model can coexist with standard interfaces for PQC key exchange schemes, making it a suitable candidate for inclusion in future quantum-safe profiles of TLS, SSH, and IPsec.

Standardization would require the publication of an official test-vector suite, formal proofs of correctness for the KMAC-based tag derivation, and reference conformance tools. Once validated, CSX could serve as a long-term symmetric complement to post-quantum key exchange mechanisms, preserving end-to-end 256-bit security across the QRCS protocol family.

16.4 Broader scientific implications

The CSX construction demonstrates that wide-word ARX permutations, when properly keyed and domain-separated through a Keccak-based sponge, can achieve AEAD functionality with minimal algorithmic complexity and strong analytical transparency. This hybrid model combines the formal rigor of permutation-based cryptography with the implementation advantages of ARX ciphers. It provides an alternative path to authenticated encryption that avoids specialized algebraic structures or complex finite-field arithmetic, reinforcing the view that post-quantum security can be realized through simplicity and compositional proofs rather than novel mathematical assumptions.

16.5 Conclusion

The CSX AEAD stream cipher achieves its stated design objectives: robust confidentiality and integrity at a post-quantum equivalent security level of 256 bits, deterministic and constant-time operation, and compatibility with standardized cryptographic components. Cryptanalytic evaluation found no exploitable weaknesses within its ARX permutation or its Keccak-based authentication mechanism. The cipher's structure provides strong margins against all known forms of classical and quantum cryptanalysis,

and its implementation adheres to modern standards for side-channel resistance and secure memory handling.

In the context of post-quantum cryptographic systems, CSX offers a mathematically grounded and operationally transparent alternative to traditional AEAD schemes. Its adoption would strengthen the long-term resilience of secure communication infrastructures against both current and emerging computational threats.

References

1. **Bertoni, G., Daemen, J., Peeters, M., and Van Assche, G.** *Keccak Specifications and Rationale*. NIST FIPS 202, 2015.
2. **Dobraunig, C., Mendel, F., and Schl  ffer, M.** *Ascon: Lightweight Authenticated Encryption and Hash Function*. IACR Transactions on Cryptographic Hardware and Embedded Systems, 2019.
3. **Bernstein, D. J., and Schwabe, P.** *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, 2018.
4. **Aumasson, J.-P.** *ARX Ciphers and the Design of Stream Cipher Permutations*. 2014.
5. **National Institute of Standards and Technology.** *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)*. SP 800-38D, 2007.
6. **National Institute of Standards and Technology.** *Recommendation for KMAC, TupleHash and ParallelHash*. SP 800-185, 2016.
7. **QRCS Corp.** *CSX Specification v1.0*. Technical Memorandum, 2025.
8. **QRCS Corp.** *QSC Cryptographic Library Reference Implementation*. Source Distribution, 2025.
9. **MISRA Consortium.** *MISRA C:2012 Guidelines for the Use of the C Language in Critical Systems*. 2013.
10. **ISO/IEC 19790:2012.** *Security Requirements for Cryptographic Modules*. International Organization for Standardization.

11. **National Institute of Standards and Technology.** *Security Requirements for Cryptographic Modules (FIPS PUB 140-3)*. 2019.
12. **Van Oorschot, P. C., and Wiener, M. J.** *Parallel Collision Search with Cryptanalytic Applications*. Journal of Cryptology, 1999.
13. **Menezes, A., van Oorschot, P., and Vanstone, S.** *Handbook of Applied Cryptography*. CRC Press, 1996.
14. **QRCS Corp.** *Quantum-Resistant Protocol Integration Guidelines*. Revision 1.2, 2025.